

第七章：中间代码生成(3)

- 过/函调用的中间代码生成
- 控制语句的中间代码生成
- 过/函声明的中间代码生成

过程调用和函数调用的中间代码

- 过程调用和函数调用的语法形式

$\text{ProcFunCall} \rightarrow \text{id} (E_1, \dots, E_n)$

- 形参种类:

值参、变量参数、函数参数

- 需要的信息:

形参的种类、传送的内容、偏移、传送的个数、函数类型(实在函数、形式函数)

过程调用和函数调用的中间代码

- $f(E_1 \dots E_n)$ 的中间代码结构:

E_1 的中间代码

.....

E_n 的中间代码

(VarACT / ValACT, t_1 , Offset $_1$, Size $_1$)

.....

(VarACT / ValACT, t_n , Offset $_n$, Size $_n$)

实参的计算
结果传递到
相应的形参
变量

(CALL, $\langle f \rangle$, true/false, Result)

调用过程/函数体执行

注: true静态确定转向地址; false:动态确定转向地址 (id为形参函数);

过程调用和函数调用的中间代码

(E_n . typ , E_n 语义信息)

.....

(E_1 . typ , E_1 语义信息)

(— , id)

要检查的语义错误:

【1】id是不是函数名

【2】每个实参 E_i 和形参 X_i 的类型和种类方面是否匹配

【3】实参个数和形参个数是否相同

例：假设有实在函数调用 $f(X+1,Y)$, 并且 x 是一般整型变量, Y 是变参整型变量, f 函数名, 同时假定 f 的两个形参第一个是值参、整数类型, 第二个是变参、整数类型, 则对应的中间代码如下:

- (**ADDI** , X , 1 , t_1)
- (**ValACT** , t_1 , Offset_1 , 1)
- (**VarACT** , Y , Offset_1+1 , 1)
- (**CALL** , f , **true** , t_2)

注: 其中 Offset_1 和 Offset_1+1 分别示表函数 f 的第1、2个参数的偏移量。

过程调用和函数调用的动作文法

$\langle \text{ProcFunCall} \rangle \rightarrow \text{id } \# \text{CallHead} \#$
 $\quad \quad \quad (\langle \text{ParamList} \rangle) \# \text{CallTail} \#$
 $\langle \text{ParamList} \rangle \rightarrow \varepsilon \mid \langle \text{ExpList} \rangle$
 $\langle \text{ExpList} \rangle \rightarrow E \# \text{EvlParam} \# \langle \text{NextList} \rangle$
 $\langle \text{NextList} \rangle \rightarrow \varepsilon \mid , \langle \text{ExpList} \rangle$

其中

- **CallHead**: 当遇到过程/函数名id时, 将其符号表地址压入Sem栈, 令实参计数器为0。
- **EvlParam**: 对每个实参 E_i : 产生它的中间代码, 将结果的类型和语义信息压入Sem栈, 实参计数器加1。

```

<ProcFunCall> → id #CallHead#
                ( <ParamList> ) #CallTail#
<ParamList> → ε | <ExpList>
<ExpList> → E #EvParam#
<NextList>
<NextList> → ε | , <ExpList>

```

➤ CallTail:

- ◆ 取出id的所有语义信息。
- ◆ 检查形、实参个数是否一致，检查形、实参类型是否相容；
- ◆ 产生送实参信息到形参信息的VaLAct/VarAct中间代码；
- ◆ 根据f是实在过程(函数)名或形式过程(函数)名产生相应的CALL代码；
- ◆ 删除当前过程/函数调用语句所占的语义栈单元，如果f是函数，则把返回值的类型和语义信息压入Sem栈。

Sem

($E_n . typ$, E_n 语义信息)

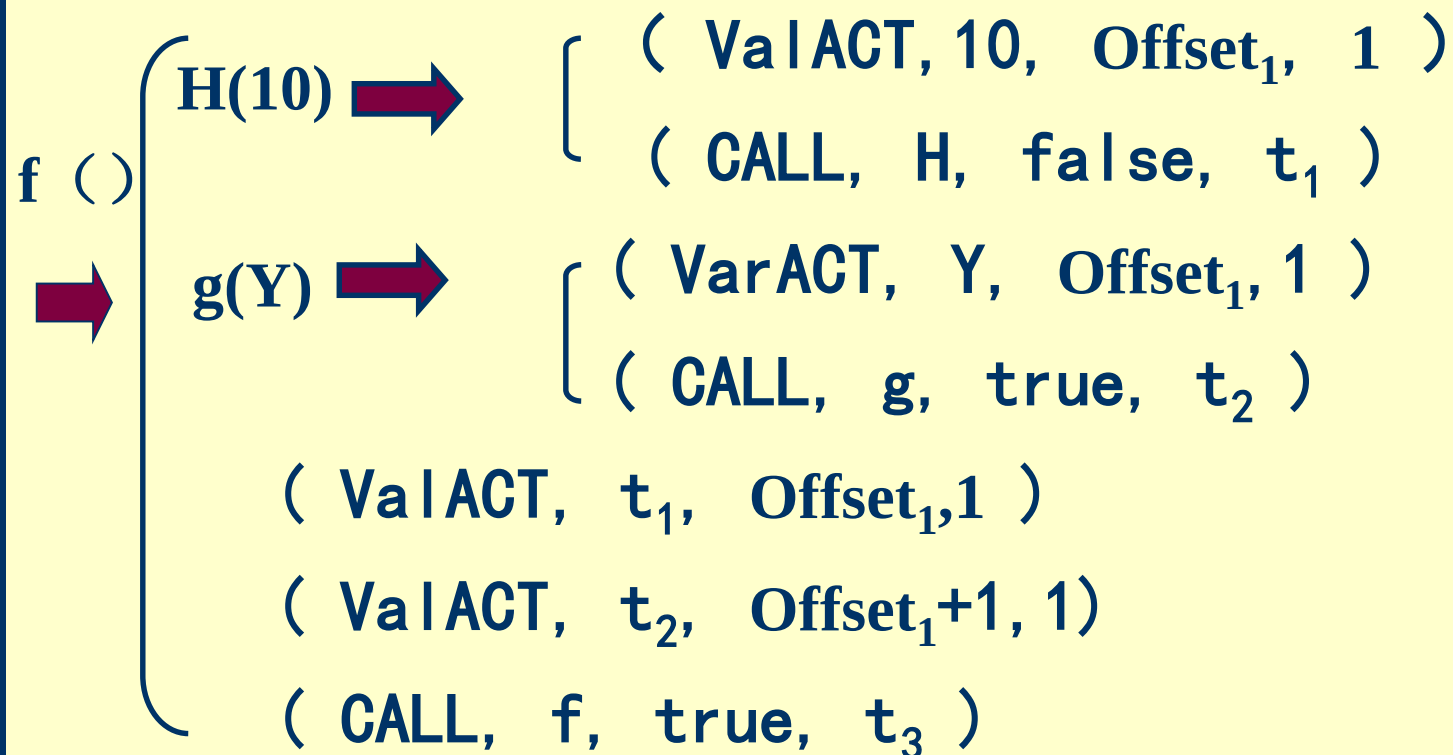
.....

($E_1 . typ$, E_1 语义信息)

(— , id)

例: $x + f(H(10), g(Y))$

其中: x 是整型变量, H 为返回值是整型的形参函数名, H 的形参为整型值参, f, g 为返回值是整型的实在函数名, f 的参数均为整型值参, g 的参数为变参。



$x + f()$ $(ADDI, x, t_3, t_4)$

注: 其中 $Offset_1$ 表示函数 H, g 及 f 的第1个参数的偏移量。

控制语句的中间代码生成

- ◆ 条件语句的中间代码
- ◆ While语句的中间代码

条件语句的中间代码

◆ $S \rightarrow \text{if } (E) \text{ then } S1 \text{ else } S2$ (1)

◆ $S \rightarrow \text{if } (E) \text{ then } S2$ (2)

(1) 的四元式结构:

E的四元式

(THEN , E. FORM , — , —) 作用: 产生条件转移

S1的四元式

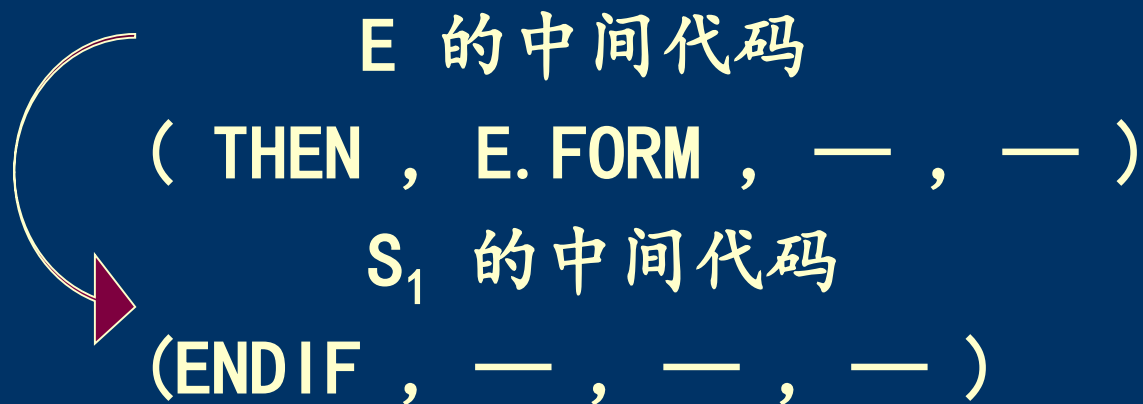
(ELSE, — , — , →) 作用: 转移、定位

S2的四元式

(ENDIF , — , — , ←) 作用: 定位

条件语句的中间代码

■ if E then S_1 (2) 的中间代码结构:



◆ 条件语句生成中间代码的动作文法：

$\langle S \rangle \rightarrow \text{if } \langle E \rangle \text{ then } \# \text{ThenIf} \# \langle S \rangle$

$\langle \text{ElsePart} \rangle \# \text{EndIf} \#$

$\langle \text{ElsePart} \rangle \rightarrow \text{else } \# \text{ElseIf} \# \langle S \rangle$

$\langle \text{ElsePart} \rangle \rightarrow \varepsilon$

- **ThenIf**: 当遇到关键字then时，该语句的条件表达式计算结果的语义信息已经在语义栈的栈顶，因此主要完成以下工作

- 1) 根据Sem[top].FORM的值，检查它的类型是否为boolean类型，
- 2) 产生中间代码：(THEN , Sem [top] , — , —)；
- 3) E弹栈：pop(1)

- **EndIf**: 遇条件语句结束符: 产生中间代码:
(ENDIF , — , — , —)
- **ElseIf**: 当遇到关键字else时, 产生中间代码 :
(ELSE , — , — , —)

While语句的中间代码

- ◆ While语句形式为：

$S \rightarrow \text{while } (E) \text{ do } S$

- ◆ While语句的中间代码结构：

(WHILE , — , — , —) 定位
E 的中间代码

(DO , E . FORM , — , —) 条件转移
S的中间代码

(ENDWHILE , — , — , —)
定位、转移

◆ while语句的中间代码生成动作文法:

$\langle S \rangle \rightarrow \text{while } \# \text{StartWhile} \# \langle E \rangle \text{ do}$
 $\quad \# \text{DoWhile} \# \langle S \rangle \# \text{EndWhile} \#$

其中动作子程序

◆ StartWhile: 遇 while 时: 产生中间代码:

(WHILE, —, —, —)

◆ DoWhile: 遇 do 时 (表达式E处理完, 其类型和地址在 Sem[top]) :

(1) 类型检查: 检查E是否为boolean类型

(2) 产生中间代码: (DO, Sem[top].FORM, —, —)

(3) E弹栈: pop(1)

◆ EndWhile: 遇循环结束符时: 产生中间代码 :

(ENDWHILE, —, —, —)

```

(:=, 1,—,a)
( WHILE , — , — , — )
(<= , a, 10, t0)
( DO , t0 , — , — )
(<> , a,b,t1)
( THEN , t1 , — , — )
( - ,a , 1 , t2 )
( * , t2 , 1 , t3 )
([ ], A , t3, t4 )
( - ,b , 1 , t5 )
( * , t5 , 1 , t6 )
([ ], A, t6, t7)
( + , t7 , 2, t8 )

```

```

a:=1;
while a<=10 do
    begin if a<>b then A[a]:=A[b]+2
           else a:=a+1;
           b:=b+1;
    end

```

```

(:=, t8,—,t4)
( ELSE , — , — , — )
( + , a , 1, t9 )
(:=, t9,—,a )
( ENDIF , — , — , — )
( + , b , 1, t10 )
(:=, t10,—,b )
(ENDWHILE ,— , — , — )

```


过程 / 函数声明的中间代码生成

- ◆ 对应过程/函数Q声明的中间代码结构:
(ENTRY , Label_Q , Size_Q , Level_Q) 或 (ENTRY,Q,-,-)
Q函数/过程体的中间代码
(ENDPROC , — , — , —) 或 (ENDFUNC , — , — , —)

过程 / 函数声明的中间代码生成

- ♦ 过程/函数声明的形式可定义为：

$\text{ProcFunDec} \rightarrow \text{ProcDec} \mid \text{FunDec}$

$\text{ProcDec} \rightarrow \text{Procedure id (ParamList)}$
 Declaration
 ProgramBody

$\text{FunDec} \rightarrow \text{Function id (ParamList) : Type}$
 Declaration
 ProgramBody

其中ParamList对应参数声明，Declaration对应整个声明部分，ProgramBody对应程序体，而Type对应类型定义。

动作文法为:

◆ $\text{ProcFunDec} \rightarrow \text{ProcDec} \mid \text{FunDec}$

$\text{ProcDec} \rightarrow$

Procedure id (ParamList) ; Declaration #Entry# ProgramBody
#EndProc#

◆ $\text{FunDec} \rightarrow$

Function id (ParamList) : Type; Declaration #Entry#
ProgramBody #EndFunc#

◆ Entry:

给子程序Q分配新标号Label_Q , 并将它填到Q的符号表项中; 产生入口中间代码:

(ENTRY , Label_Q , Size_Q , Level_Q) 或 (ENTRY , Q , - , -)

◆ EndProc和EndFunc:

产生出口中间代码:

(ENDPROC , — , — , —) 或 (ENDFUNC , — , — , —)

例：过程声明的中间代码（不保存符号表）

```
procedure Q( x: real );  
  var u : real ;  
  function f( k: real ):real;  
    begin  
      f := k +k  
    end;  
begin  
  u := f(50);  
  y:= u * x  
end;
```

```
(ENTRY, Labelf, Sizef, Lf)  
(ADDF, k, k, t0)  
(:=, t0, -, f)  
(ENDFUNC, _ , _ , _ )  
(ENTRY, LabelQ, SizeQ, LQ)  
(FLOAT, 50, _ , t1)  
(VALACT, t1, offk, 2)  
(CALL, Labelf, true, t2)  
(:=, t2, -, u)  
(MULTF, u, x, t3)  
(:=, t3, -, y)  
(ENDPROC, _ , _ , _ )
```